

Rockchip RV1126B Linux SDK Quick Start Guide

Document Identifier: RK-JC-YF-A47

Release Version: V1.1.0

Date: 2025-09-20

Security Level: ☐Top-Secret ☐Secret ☐Internal ☒Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED "AS IS". ROCKCHIP ELECTRONICS CO., LTD. ("ROCKCHIP") DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025 Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document primarily describes the basic usage of theRV1126B Linux SDK, aiming to help developers quickly understand and use the RV1126B SDK development package.

Target Audience

This document (this guide) is mainly applicable to the following engineers:

Technical Support Engineers

Software Development Engineers

Chip System Support Status

Chip Name	Uboot Version	Kernel Version	Buildroot Version	Yocto Version	Debian Version
RV1126B, RV1126B-P	2017.9	6.1	2024.02	5.0	12

Revision Record

Date	Version	Author	Modification Description
2025-03-26	V0.0.1	Caesar Wang	Alpha version
2025-05-15	V0.1.0	Caesar Wang	Beta version
2025-06-20	V1.0.0	Caesar Wang	Release version
2025-09-20	V1.1.0	Caesar Wang	1. Update Configuration Item Description 2. Add AVS Description 3. Add AI-ISP Description 4. Add AI-Remosaic Description 5. Add FEC Description 6. Add How to Read CPU Serial Number

Table of Contents

Rockchip RV1126B Linux SDK Quick Start Guide

1. Precompiled SDK Images
2. Development Environment Setup
 - 2.1 Preparing the Development Environment
 - 2.2 Installing Libraries and Toolkits
 - 2.2.1 Checking and Upgrading the Host's Python Version
 - 2.2.2 Checking and Upgrading the Host's `make` Version
 - 2.2.3 Checking and Upgrading the Host's LZ4 Version
3. Docker Environment Setup
4. Software Development Guide
 - 4.1 Development Guide
 - 4.2 Buildroot Development Guide
 - 4.3 Debian Development Guide
 - 4.4 Third-Party OS Porting
 - 4.5 MCU Development Guide
 - 4.6 RKNPU Development Guide
 - 4.6.1 RKNN-TOOLKIT2
 - 4.6.2 RKNN Runtime
 - 4.6.3 RKNN Driver
 - 4.6.4 RKNN LLM
 - 4.7 Real-Time Linux Development Notes
 - 4.7.1 PREEMPT_RT Patch
 - 4.7.2 Xenomai Cobalt Mode
 - 4.8 RKIPC Developer Guide
 - 4.9 AVS Multi-View Stitching
 - 4.10 AI-ISP Image Enhancement
 - 4.11 AI-Remosaic Pixel Rearrangement
 - 4.12 FEC Fisheye Distortion Correction
 - 4.13 Software Update Log
5. Hardware Development Guide
 - 5.1 Chip Documentation
 - 5.2 Hardware Documentation
6. SDK Project Directory Overview
7. Introduction to Cross-Compilation Toolchain for SDK
 - 7.1 U-Boot and Kernel Compilation Toolchain
 - 7.2 Buildroot Toolchain
 - 7.2.1 Configuring the Compilation Environment
 - 7.2.2 Packaging Toolchain
 - 7.3 Debian Toolchain
 - 7.4 Yocto Toolchain
8. SDK Compilation Instructions
 - 8.1 Viewing SDK Compilation Commands
 - 8.2 SDK Board-Level Configuration
 - 8.3 SDK Custom Configuration
 - 8.4 Fully Automatic Compilation
 - 8.5 Module Compilation
 - 8.5.1 U-Boot Compilation
 - 8.5.2 Kernel Compilation
 - 8.5.3 Recovery Compilation
 - 8.5.4 Buildroot Compilation

- 8.5.4.1 Buildroot Module Compilation
 - 8.5.5 Debian Compilation
 - 8.5.6 Yocto Compilation
 - 8.6 Firmware Packaging
- 9. Flashing Instructions
 - 9.1 Windows Flashing Instructions
 - 9.2 Linux Flashing Instructions
 - 9.3 System Partition Description
- 10. Frequently Asked Questions (FAQ)
 - 10.1 How to Enable DDR Frequency Scaling
 - 10.2 Issue with RV1126BP Read/Write
 - 10.3 How to Read CPU Serial Number

1. Precompiled SDK Images

Developers can bypass the process of compiling the entire operating system from source code by using the precompiled RV1126B Linux SDK image, which allows for a quick start in development and related assessments and comparisons. This can reduce the waste of development time and cost due to compilation issues.

The firmware can be downloaded from the public address [Click here for SDK firmware download](#).

Firmware path: **Universal Linux SDK Firmware -> Linux 6.1 -> RV1126B**

For those who need to modify the SDK code or require a quick start, please refer to the following sections.

2. Development Environment Setup

2.1 Preparing the Development Environment

We recommend using a system with Ubuntu 22.04 or a higher version for compilation. Other Linux versions may require corresponding adjustments to the software packages. In addition to system requirements, there are other hardware and software requirements.

Hardware Requirements: 64-bit system with hard disk space greater than 40GB. If you are performing multiple builds, you will need more hard disk space.

Software Requirements: System with Ubuntu 22.04 or a higher version.

2.2 Installing Libraries and Toolkits

When developing for devices using the command line, you can install the necessary libraries and tools required for compiling the SDK by following these steps.

Use the following `apt-get` commands to install the libraries and tools needed for subsequent operations:

```
sudo apt-get update && \
sudo apt-get install git ssh make gcc libssl-dev liblz4-tool expect \
expect-dev g++ patchelf chrpath gawk texinfo chrpath \
diffstat binfmt-support qemu-user-static live-build bison flex \
fakeroot cmake gcc-multilib g++-multilib unzip device-tree-compiler \
ncurses-dev libgucharmap-2-90-dev bzip2 expat gpgv2 cpp-aarch64-linux-gnu \
libgmp-dev libmpc-dev bc python-is-python3 python2 gettext libc6-dev \ libncurses-dev rsync
```

Note:

The installation command is suitable for Ubuntu 22.04. For other versions, please use the corresponding installation command based on the package name. If you encounter errors during compilation, you can install the corresponding software packages based on the error messages. Specifically:

- Python requires the installation of version 3.6 or higher, with version 3.6 used as an example here.
- Make requires the installation of version 4.0 or higher, with version 4.2 used as an example here.
- LZ4 requires the installation of version 1.7.3 or higher.
- Compiling Yocto requires a VPN network.

2.2.1 Checking and Upgrading the Host's Python Version

The method to check and upgrade the host's Python version is as follows:

- Check the host's Python version

```
$ python3 --version
Python 3.10.6
```

If the requirement of Python version ≥ 3.6 is not met, you can upgrade by the following method:

- Upgrade to the new version of Python 3.6.15

```
PYTHON3_VER=3.6.15
echo "wget https://www.python.org/ftp/python/${PYTHON3_VER}/Python-${PYTHON3_VER}.tgz"
echo "tar xf Python-${PYTHON3_VER}.tgz"
echo "cd Python-${PYTHON3_VER}"
echo "sudo apt-get install libsqlite3-dev"
echo "./configure --enable-optimizations"
echo "sudo make install -j8"
```

2.2.2 Checking and Upgrading the Host's make Version

The method to check and upgrade the host's make version is as follows:

- Checking the host's make version

```
$ make -v
GNU Make 4.2
Built for x86_64-pc-linux-gnu
```

- Upgrading to a newer version of make 4.2

```
$ sudo apt update && sudo apt install -y autoconf autopoint

git clone https://gitee.com/mirrors/make.git
cd make
git checkout 4.2
git am $BUILDR00T_DIR/package/make/*.patch
autoreconf -f -i
./configure
make make -j8
sudo install -m 0755 make /usr/bin/make
```

2.2.3 Checking and Upgrading the Host's LZ4 Version

The method to check and upgrade the host's LZ4 version is as follows:

- Check the host's LZ4 version

```
$ lz4 -v
*** LZ4 command line interface 64-bits v1.9.3, by Yann Collet ***
```

- Upgrade to a new version of LZ4

```
git clone https://gitee.com/mirrors/LZ4_old1.git
cd LZ4_old1

make
sudo make install
sudo install -m 0755 lz4 /usr/bin/lz4
```

3. Docker Environment Setup

To assist developers in quickly completing the complex preparation of the development environment mentioned above, we also provide a cross-compiler Docker image for clients to rapidly verify, thereby reducing the time required to build the compilation environment.

Before using the Docker environment, you can refer to the following document for operations:

https://<SDK>/docs/en/Linux/Docker/Rockchip_Developer_Guide_Linux_Docker_Deploy_EN.pdf.

The verified systems are as follows:

Distribution Version	Docker Version	Image Loading	Firmware Compilation
Ubuntu 22.04	24.0.5	pass	pass
Ubuntu 21.10	20.10.12	pass	pass
Ubuntu 21.04	20.10.7	pass	pass
Ubuntu 18.04	20.10.7	pass	pass
Fedora 35	20.10.12	pass	NR (not run)

The Docker image can be obtained from the URL [Docker Image](#).

4. Software Development Guide

4.1 Development Guide

To help developers quickly become familiar with the SDK development and debugging process, the development guide titled `Rockchip_Developer_Guide_Linux_Software_EN.pdf` is released in synchronization with the SDK. This guide covers the entire process, including chip support status, roadmap, download and software package acquisition, development documentation/tools/framework/environment setup/version management/compilation/partition configuration/firmware upgrade/module development/debugging tool description, software and licensing/open-source information, frequently asked questions, and permission application.

It can be obtained in the `docs/en/RV1126B` directory and will be continuously improved and updated.

4.2 Buildroot Development Guide

To assist developers in quickly getting started with the development and debugging of the Buildroot system, the development guide "Rockchip_Developer_Guide_Buildroot_EN.pdf" is released along with the SDK. It can be obtained in the `docs/en/Linux/System` directory and will be continuously improved and updated.

4.3 Debian Development Guide

To assist developers in quickly becoming proficient with the development and debugging of the Debian system, the SDK release includes the "Rockchip_Developer_Guide_Debian_EN.pdf" development guide, which can be obtained under `docs/en/Linux/System` and will be continuously improved and updated.

4.4 Third-Party OS Porting

To assist development engineers in quickly mastering the porting and adaptation of third-party operating systems, the SDK release includes the development guide titled "Rockchip_Developer_Guide_Third_Party_System_Adaptation_EN.pdf". It can be obtained in the `docs/en/Linux/System` directory and will be continuously improved and updated.

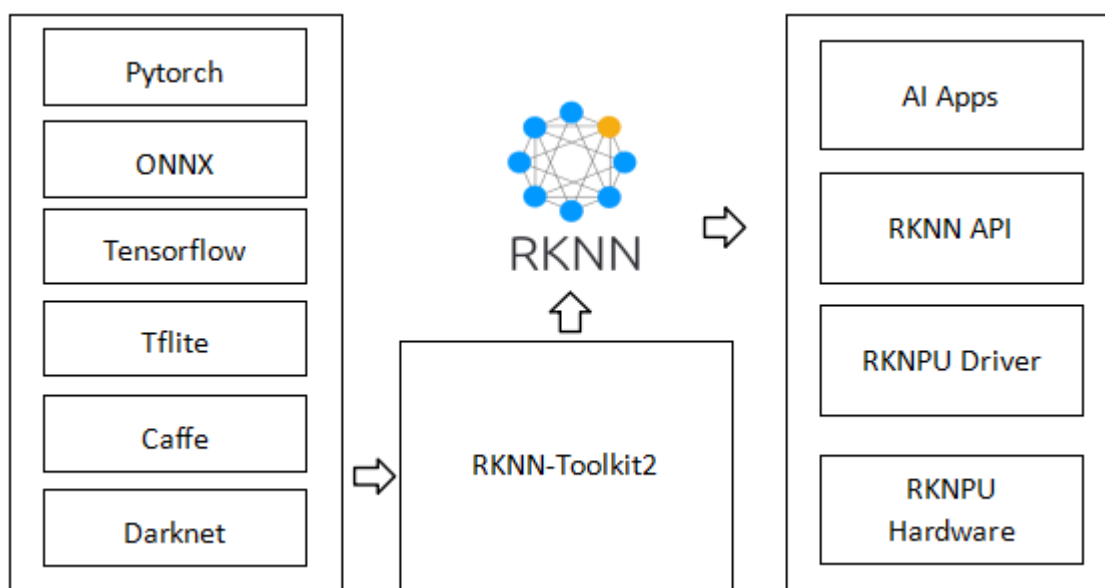
4.5 MCU Development Guide

For the basic usage of the RV1126B MCU, please refer to the following document:

```
<SDK>/docs/en/RV1126B/Quick-start/Rockchip_RV1126B_Quick_Start_RT-Thread_MCU_EN.pdf
```

4.6 RKNPU Development Guide

RKNN software stack can help users to quickly deploy AI models to Rockchip chips. The overall framework is as follows:



In order to use RKNPU, users need to first run the RKNN-Toolkit2 tool on the computer, convert the trained model into an RKNN format model, and then inference on the development board using the RKNN C API or Python API.

4.6.1 RKNN-TOOLKIT2

RKNN-Toolkit2 is a software development kit for users to perform model conversion, inference and performance evaluation on PC and Rockchip NPU platforms.

RKNN-Toolkit-Lite2 provides Python programming interfaces for Rockchip NPU platform to help users deploy RKNN models and accelerate the implementation of AI applications.

The development kit is located in the `external/rknn-toolkit2` directory, mainly used to implement a series of functions such as model conversion, optimization, quantization, inference, performance evaluation, and accuracy analysis.

Basic functions are as follows:

Function	Description
Model Conversion	Supports floating-point models of Pytorch / TensorFlow / TFLite / ONNX / Caffe / Darknet Supports quantization-aware models (QAT) of Pytorch / TensorFlow / TFLite Supports dynamic input models (dynamicization/native dynamic) Supports large models
Model Optimization	Constant folding / OP correction / OP Fuse&Convert / Weight sparsification / Model pruning
Model Quantization	Supports quantization types: asymmetric i8/fp16 Supports Layer / Channel quantization methods; Normal / KL/ MMSE quantization algorithms Supports mixed quantization to balance performance and accuracy
Model Inference	Supports model inference through the simulator on the PC Supports model inference on the NPU hardware platform (board-level inference) Supports batch inference, supports multi-input models
Model Evaluation	Supports performance and memory evaluation of the model on the NPU hardware platform
Accuracy Analysis	Supports quantization accuracy analysis function (simulator/NPU)
Additional Features	Supports version/device query functions, etc.

4.6.2 RKNN Runtime

The development documentation for RKNN Runtime is located in the `external/rknpu2` directory of the project. It is used to infer models generated by RKNN-Toolkit2 and provides C/C++ programming interfaces for the Rockchip NPU platform, assisting users in deploying RKNN models and accelerating AI application implementation.

For detailed usage instructions, refer to the documentation in the current `doc/` directory:

```

└─ 01_Rockchip_RKNPU_Quick_Start_RKNN_SDK_V2.3.2_EN.pdf
└─ 02_Rockchip_RKNPU_User_Guide_RKNN_SDK_V2.3.2_EN.pdf
└─ 03_Rockchip_RKNPU_API_Reference_RKNN_Toolkit2_V2.3.2_EN.pdf
└─ 04_Rockchip_RKNPU_API_Reference_RKNNRT_V2.3.2_EN.pdf
└─ 05_RKNN_Compiler_Support_Operator_List_V2.3.2.pdf
└─ RKNNToolkit2_API_Difference_With_Toolkit1-2.3.2.md
└─ RKNNToolkit2_OP_Support-2.3.2.md
...

```

4.6.3 RKNN Driver

The RKNN kernel driver handles interactions with NPU hardware and is integrated into the Rockchip kernel code under `drivers/rknpu`.

4.6.4 RKNN LLM

RKNN LLM is an inference framework optimized by Rockchip for large language model (LLM) scenarios. It aims to help users efficiently deploy and run large pre-trained language models (e.g., LLaMA, PaLM, ChatGLM) on Rockchip's proprietary NPU hardware platform.

For detailed usage instructions, refer to the documentation in the current `doc/` directory:

```

└─ Rockchip_RKLLM_SDK_EN_1.2.0.pdf

```

- RKLLM Large-Model Test

By default, the build must enable `#include "ai/rkai.config"`

Example patch for `configs/rockchip_rv1126b_defconfig`:

```

--- a/configs/rockchip_rv1126b_defconfig
+++ b/configs/rockchip_rv1126b_defconfig
@@ -13,6 +13,7 @@
 #include "multimedia/gst/rtsp.config"
 #include "multimedia/gst/video.config"
 #include "multimedia/mpp.config"
+#include "ai/rkai.config"
 #include "ai/npu2.config"

```

After rebuilding the image, run the VLM demo with the bundled config:

```
rkai_VLM_Demo /usr/share/app_conf/config/rkai_vlm_fastvlm_rv1126b.json
```

```

buildroot# rkai_VLM_Demo /usr/share/app_conf/config/rkai_vlm_fastvlm_rv1126b.json
rkai vlm version: 0.0.1
=====Config Loaded:=====
vision_model_path: /usr/share/app_conf/models/fastvlm-0.5b-fp16.rv1126b.rknn
llm_model_path: /usr/share/app_conf/models/fastvlm-0.5B-w8a8_level0.rv1126b.rkllm

```

```

    input_path: /usr/share/app_conf/inputs/image_1024x1204.npy
    max_context_len: 512
    max_new_tokens: 128
        top_k: 1
skip_special_token: 1
    base_domain_id:0
        img_start:<|im_start|>
        img_end:<|im_end|>
        img_content:<|im_content|>
=====
model input num: 1, output num: 1
===== Inputs Attribute =====
Index      : 0
Name       : images
Num Dims   : 4
Dims      : [1, 1024, 1024, 3]
Num Elems  : 3145728
Size       : 6291456 bytes
Format     : NHWC
Type       : FP16
Qnt Type   : AFFINE
Zero Point : 0
Scale      : 1.000000
=====

===== Outputs Attribute =====
Index      : 0
Name       : projector_features
Num Dims   : 3
Dims      : [1, 256, 896]
Num Elems  : 229376
Size       : 458752 bytes
Format     : UNDEFINED
Type       : FP16
Qnt Type   : AFFINE
Zero Point : 0
Scale      : 1.000000
=====

model input height=1024, width=1024, channel=3,out_elems=229376
main: Model Vision loaded in 2329.10 ms
I rkllm: rkllm-runtime version: 1.2.2, rknpu driver version: 0.9.8, platform: RV1126B
I rkllm: loading rkllm model from /usr/share/app_conf/models/fastvlm-0.5B-
w8a8_level0.rv1126b.rkllm
I rkllm: rkllm-toolkit version: 1.2.1, max_context_limit: 4096, npu_core_num: 1,
target_platform: RV1126B, model_dtype: W8A8
I rkllm: Enabled cpus: [0, 1, 2, 3]
I rkllm: Enabled cpus num: 4
rkllm init success
main: Model LLM loaded in 4722.97 ms
Loading /usr/share/app_conf/inputs/image_1024x1204.npy
numpy data type:uint8
rkai_ctx->img_elems: 229376

```

```
main: Model run in 2744.84 ms
```

```
*****可输入以下问题对应序号获取回答/或自定义输入*****
```

```
[0] <image>What is in the image?
```

```
[1] <image>这张图片中有什么元素?
```

```
*****
```

```
user: 0
```

```
<image>What is in the image?
```

```
I rkllm: reset chat template:
```

```
I rkllm: system_prompt: <|im_start|>system\nYou are a helpful assistant.<|im_end|>\n
```

```
I rkllm: prompt_prefix: <|im_start|>user\n
```

```
I rkllm: prompt_postfix: <|im_end|>\n<|im_start|>assistant\n
```

```
W rkllm: Calling rkllm_set_chat_template will disable the internal automatic chat template  
parsing, including enable_thinking. Make sure your custom prompt is complete.
```

```
user: <image>What is in the image?
```

```
robot: In the image, there is an astronaut on the moon.
```

```
+-----+-----+  
| Stage           | Perf               |  
+-----+-----+  
| Vision          | 2745               (ms) |  
| TTFT            | 1682.863           (ms) |  
| TPS             | 13.074 (tokens/s) |  
+-----+-----+
```

4.7 Real-Time Linux Development Notes

As the demand for real-time performance in products increases, the real-time capabilities of standard Linux are no longer sufficient for many applications. It is necessary to optimize standard Linux to enhance its real-time performance, such as using PREEMPT_RT or the Xenomai real-time system.

Below are the latency results from stress tests conducted on RV1126B Buildroot using PREEMPT_RT and Xenomai, respectively:

Stress Test:

```
stress-ng -c 4 --io 2 --vm 2 --vm-bytes 256M --timeout 1000000s
```

4.7.1 PREEMPT_RT Patch

```
root@rv1126b-buildroot:/# cyclictest -c 0 -m -a -t 4 -p 99 -D 24h

Apolicy: fifo: loadavg: 10.80 11.74 12.03 11/232 3403

T: 0 ( 1915) P:99 I:1000 C:82647713 Min:      0 Act:      3 Avg:      2 Max:      38
T: 1 ( 1916) P:99 I:1000 C:82647712 Min:      0 Act:      2 Avg:      2 Max:      43
T: 2 ( 1917) P:99 I:1000 C:82647709 Min:      0 Act:      3 Avg:      2 Max:      47
T: 3 ( 1918) P:99 I:1000 C:82647698 Min:      0 Act:      3 Avg:      3 Max:      58
(Tested for 24 hours)
```

4.7.2 Xenomai Cobalt Mode

```
root@rv1126b-buildroot:/# echo 0 > /proc/xenomai/latency
root@rv1126b-buildroot:/# ./usr/demo/cyclictest -m -n -c 0 -p99 -t -D 24h

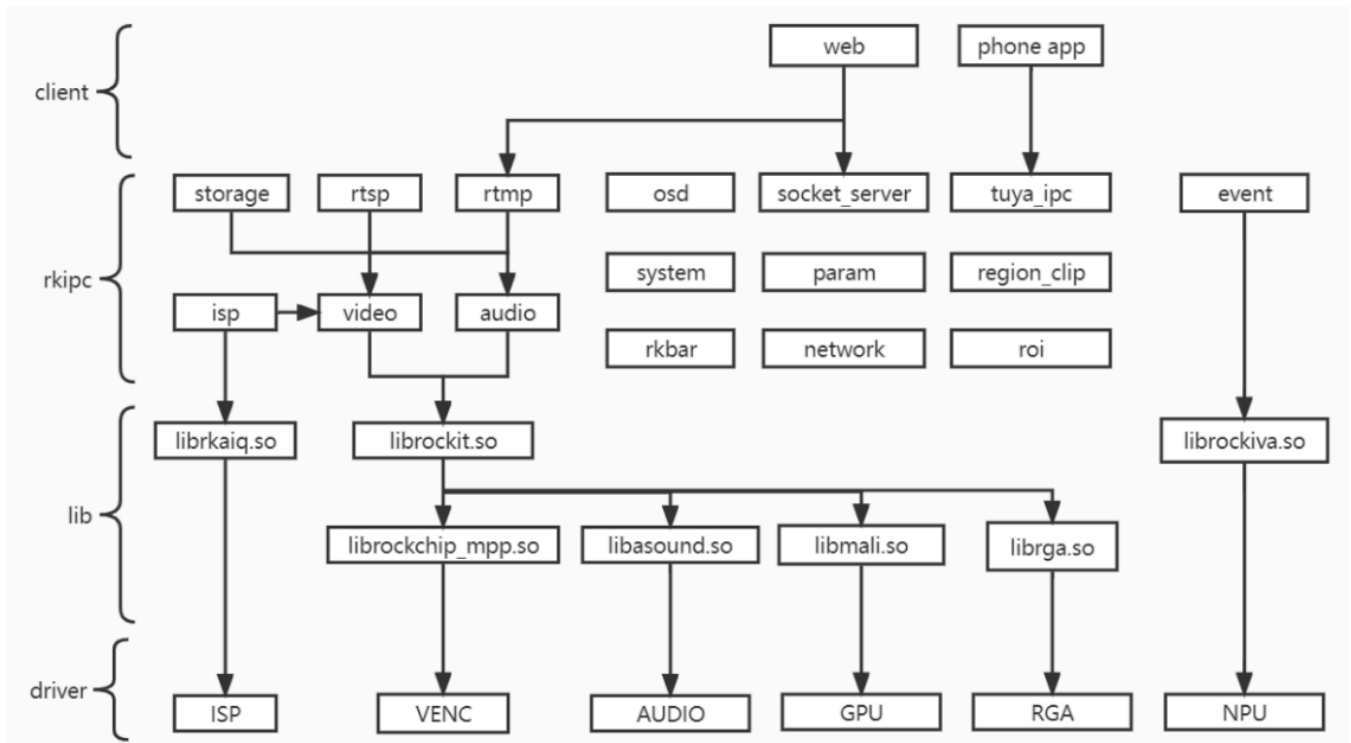
Apolicy: fifo: loadavg: 9.06 8.88 8.84 9/166 3460

T: 0 ( 1837) P:99 I:1000 C:86399995 Min:      2 Act:      6 Avg:      7 Max:      91
T: 1 ( 1838) P:99 I:1000 C:86399979 Min:      2 Act:      8 Avg:      8 Max:      70
T: 2 ( 1839) P:99 I:1000 C:86399963 Min:      2 Act:      8 Avg:      6 Max:      41
T: 3 ( 1840) P:99 I:1000 C:86399939 Min:      2 Act:      8 Avg:      9 Max:      72
(Tested for 24 hours)
```

For more details, please refer to the development patches and documentation in [docs/Patches/Real-Time-Performance/](#).

4.8 RKIPC Developer Guide

This guide targets single-channel IPC products on the RV1126B platform, supporting web interfaces, RTSP/RTMP preview, and dynamic parameter adjustment. The overall architecture of RKIPC is as follows:



For the RKIPC application development guide, refer to docs/en/Linux/ApplicationNote/Rockchip_Developer_Guide_Linux_RKIPC_EN.pdf.

4.9 AVS Multi-View Stitching

The AVS (Any View Stitching) algorithm supports real-time stitching of multiple input images into a single image, effectively expanding the overall field of view.

- **API Reference:** Refer to the AVS chapter in docs/en/Socs/RV1126B/RKIPC/Rockchip_Developer_Guide_MPI_C_EN.pdf.
- **Calibration Tool Download:** <https://console.zbox.filez.com/l/g0a8F8>;

4.10 AI-ISP Image Enhancement

AI-ISP optimizes image quality based on deep learning, covering noise suppression, clarity, contrast, brightness, and color enhancement.

The RV1126B integrates dedicated AI-ISP hardware, breaking through the computational bottlenecks of traditional solutions without occupying general NPU resources.

- **Required Files:** Configuration and model files in the `<SDK>/external/camera_engine_rkaiq/rkaiq/iqfiles/isp35/ainr` directory.
- **API and Debugging Reference:** Refer to the aibnr chapter in docs/en/Socs/RV1126B/RKIPC/Rockchip_Tuning_Guide_ISP35_EN.pdf.

4.11 AI-Remosaic Pixel Rearrangement

Through dedicated AI-ISP hardware, Quad Bayer/4CELL sensor data is converted into traditional Bayer arrays in real-time, enabling adaptive high-definition imaging for both day and night modes.

- **Required Files:** Configuration and model files in
`<SDK>/external/camera_engine_rkaiq/rkaiq/iqfiles/isp35/airms.`
- **Sensor Support:** Requires support for 4CELL format output. Refer to
`<SDK>/kernel/drivers/media/i2c/os12d40.c.`

4.12 FEC Fisheye Distortion Correction

Used to correct image distortion caused by fisheye lenses, restoring images closer to the human eye's perspective.

- **Development References:**
GDC chapter in `docs/en/Socs/RV1126B/RKIPC/Rockchip_Developer_Guide_MPI_C_EN.pdf`

4.13 Software Update Log

- Software release version upgrades can be viewed through the project XML. The specific method is as follows:

```
.repo/manifests$ realpath rv1126b_linux6.1_release.xml
# For example: The printed version number is v1.0.0, and the update time is 20250620
# <SDK>/ .repo/manifests/release/rv1126b_linux6.1_release_v1.0.0_20250620.xml
```

- The content of the software release version upgrades can be viewed through the project text, refer to the project directory:

```
<SDK>/ .repo/manifests/RV1126B_Linux6.1_SDK_Note.md
or
<SDK>/docs/en/RV1126B/RV1126B_Linux6.1_SDK_Note.md
```

- The board can obtain version information in the following way:

```
buildroot:/# cat /etc/os-release
NAME=Buildroot
VERSION=linux-6.1-stan-rkr6
ID=buildroot
VERSION_ID=2024.02
PRETTY_NAME="Buildroot 2024.02"
OS="buildroot"
RK_BUILD_INFO="XXX Mon Jun 16 17:11:18 CST 2025 - rockchip_rv1126b_ipc"
```


5. Hardware Development Guide

5.1 Chip Documentation

To help development engineers quickly get started with the development and debugging of the RV1126B, the SDK includes the following chip manuals: "RV1126B_brief_datasheet_V1.0_20241219.pdf" and "Rockchip_RV1126B_Datasheet_*.pdf".

These documents can be obtained in the `docs/en/RV1126B/Datasheet` directory and will be continuously improved and updated.

5.2 Hardware Documentation

Hardware-related development can refer to the user guides in the project directory

`<SDK>/docs/en/RV1126B/Hardware/ :`

```
├─ RV1126&RV1126B-  
P_Differences_Description_and_Compatibility_Design_Notes_V1.26_20250604_CN&EN.pdf  
├─ RV1126&RV1126B_Function_Mapping_Table_V1.0_20250527.xlsx  
├─ RV1126B-P_Package_PinLength_V1.0_20250318_EN.xlsx  
├─ RV1126B-P_PinOut_V1.0_20250519_EN.xlsx  
├─ RV1126B_EVB1_User_Guide_V1.0_20250623_EN.pdf  
├─ RV1126B_Methods_for_Processing_Unused_Pins_V1.0_20250606_EN.xlsx  
├─ RV1126B_Package_PinLength_V1.0_20250318_EN.xlsx  
└─ RV1126B_PinOut_V1.1_20250519_EN.xlsx
```

6. SDK Project Directory Overview

The SDK project directory includes directories such as buildroot, debian, yocto, app, kernel, u-boot, device, docs, external, prebuilts, etc. It uses a manifest to manage the repository and the repo tool to manage each directory or its corresponding git projects.

- app: Contains upper-layer application APPs, mainly some application demos.
- buildroot: The root file system developed based on Buildroot (2024).
- debian: The root file system developed based on Debian bookworm (12).
- device/rockchip: Contains chip-level board configurations as well as scripts and files for SDK compilation and firmware packaging.
- docs: Contains general development guidance documents, chip platform-related documents, Linux system development-related documents, and other reference documents.
- external: Contains third-party related repositories, including NPU, algorithms, audio/video, camera, network, security, etc.

- kernel: Contains the code for Kernel development.
- output: Contains information about each firmware generation, compilation information, XML, host environment, etc.
- prebuilts: Contains cross-compilation toolchains.
- rkbin: Contains Rockchip-related binaries and tools.
- rockdev: Contains compiled output firmware, actually a soft link to `output/firmware`.
- tools: Contains commonly used tools for Linux and Windows operating systems.
- u-boot: Contains U-Boot code developed based on the v2017.09 version.
- yocto: Contains the root file system developed based on Yocto scarthgap(5.0).

7. Introduction to Cross-Compilation Toolchain for SDK

Considering that the Rockchip Linux SDK is currently compiled only in the Linux PC environment, we have also provided the cross-compilation toolchain for Linux only. The prebuilt toolchain in the prebuilt directory is for use with U-Boot and Kernel. Specific Rootfs requires the corresponding toolchain for each, or a third-party toolchain for compilation.

7.1 U-Boot and Kernel Compilation Toolchain

The SDK prebuilts directory contains cross-compilation tools currently used for U-Boot and Kernel compilation, as follows:

Directory	Description
prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu	GCC ARM 10.3.1 64-bit toolchain
prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi	GCC ARM 10.3.1 32-bit toolchain

You can download the toolchain from the following address:

[Click here](#)

7.2 Buildroot Toolchain

7.2.1 Configuring the Compilation Environment

To compile individual modules or third-party applications, a cross-compilation environment must be set up. For instance, for the RV1126B, the cross-compilation tools are located in the `buildroot/output/rockchip_rv1126b/host/usr` directory. It is necessary to set the `bin/` directory of the tools

and the `aarch64-buildroot-linux-gnu/bin/` directory as environment variables. Execute the script for automatic configuration of the environment variables in the top-level directory:

```
source buildroot/envsetup.sh rockchip_rv1126b
```

Enter the command to check:

```
cd buildroot/output/rockchip_rv1126b/host/usr/bin  
./aarch64-linux-gcc --version
```

At this point, the following information will be printed:

```
aarch64-linux-gcc.br_real (Buildroot -gc307c95550) 13.3.0
```

7.2.2 Packaging Toolchain

Buildroot supports the packaging of the built-in toolchain into a compressed archive for standalone compilation by third-party applications. For detailed information on how to package the toolchain, please refer to the Buildroot official documentation:

```
buildroot/docs/manual/using-buildroot-toolchain.txt
```

In the SDK, you can directly run the following command to generate the toolchain package:

```
./build.sh bmake: sdk
```

The generated toolchain package is located in the `buildroot/output/*/images/` directory, named `aarch64-buildroot-linux-gnu_sdk-buildroot.tar.gz`, for users who have the need. After extraction, the path to `gcc` will be:

```
./aarch64-buildroot-linux-gnu_sdk-buildroot/bin/aarch64-buildroot-linux-gnu-gcc
```

You can download the toolchain from the following address:

[Click here](#)

7.3 Debian Toolchain

Utilize Docker on the machine side, `gcc`, or `dpkg-buildpackage` for the relevant compilation.

7.4 Yocto Toolchain

Refer to the following:

- [Building your own recipes from first principles](#)
- [New Recipe](#)

8. SDK Compilation Instructions

The SDK can be configured and compiled for specific functionalities using `make` or `./build.sh` with target arguments. For detailed instructions, refer to the compilation guide in `device/Rockchip/common/README.md`.

To ensure a smooth update process for the SDK each time, it is recommended to clean the previous compilation artifacts before updating. This practice helps to avoid potential compatibility issues or compilation errors that may arise from using old compilation artifacts with a new version of the SDK. To clean these artifacts, simply run the command `./build.sh cleanall`.

One-Click Compilation

Chip	Type	ARM Architecture Version	Permission Type	Reference Board	One-Click Compilation	rootfs Compilation	kernel Compilation	uboot Compilation
RV1126B	Standard	64-bit	RW	RV1126B EVB1	<code>./build.sh lunch:rockchip_rv1126b_evb1_v10_defconfig && ./build.sh</code>	<code>./build.sh rootfs</code>	<code>make ARCH=arm64 rockchip_linux_defconfig rv1126b.config; make ARCH=arm64 rv1126b-evb1-v10.img BOOT_ITS=boot.its -j24</code>	<code>./make.sh rv1126b --spl-new</code>
RV1126B	Industrial	64-bit	RW	RV1126B EVB4	<code>./build.sh lunch:rockchip_rv1126b_evb4_v10_defconfig && ./build.sh</code>	<code>./build.sh rootfs</code>	<code>make ARCH=arm64 rockchip_linux_defconfig rv1126b.config; make ARCH=arm64 rv1126b-evb4-v10.img BOOT_ITS=boot.its -j24</code>	<code>./make.sh rv1126b --spl-new</code>
RV1126B	Fastboot	64-bit	RO	RV1126B EVB1	<code>./build.sh lunch:rockchip_rv1126b_fastboot_defconfig && ./build.sh</code>	<code>./build.sh rootfs</code>	<code>make ARCH=arm64 rv1126b_defconfig rv1126b.config rv1126b-fastboot.config; make ARCH=arm64 rv1126b-evb1-v10-fastboot-emmc.img BOOT_ITS=boot.its -j24</code>	<code>./make.sh rv1126b-emmc-fastboot rk-gpt --spl-new</code>
RV1126B	IPC	32-bit	RW	RV1126B EVB1	<code>./build.sh lunch:rockchip_rv1126b_ipc_32_evb1_v10_defconfig && ./build.sh</code>	<code>./build.sh rootfs</code>	<code>make ARCH=arm rv1126b_linux_defconfig; make ARCH=arm rv1126b-evb1-v10.img BOOT_ITS=boot.its -j24</code>	<code>./make.sh rv1126b --spl-new</code>
RV1126B	IPC	64-bit	RW	RV1126B EVB1	<code>./build.sh lunch:rockchip_rv1126b_ipc_64_evb1_v10_defconfig && ./build.sh</code>	<code>./build.sh rootfs</code>	<code>make ARCH=arm64 rv1126b_defconfig rv1126b.config; make ARCH=arm64 rv1126b-evb1-v10.img BOOT_ITS=boot.its -j24</code>	<code>./make.sh rv1126b --spl-new</code>
RV1126B	Robot	64-bit	RW	RV1126B EVB1	<code>./build.sh lunch:rockchip_rv1126b_robot_defconfig && ./build.sh</code>	<code>./build.sh rootfs</code>	<code>make ARCH=arm64 rv1126b_defconfig rv1126b.config rv1126b_robot.config rockchip_amp.config; make ARCH=arm64 rv1126b-evb1-v10-amp.img BOOT_ITS=boot.its -j24</code>	<code>./make.sh rv1126b rk-amp --spl-new</code>
RV1126B-P	Standard	64-bit	RW	RV1126BP EVB1	<code>./build.sh lunch:rockchip_rv1126bp_evb1_v10_defconfig && ./build.sh</code>	<code>./build.sh rootfs</code>	<code>make ARCH=arm64 rockchip_linux_defconfig rv1126b.config; make ARCH=arm64 rv1126bp-evb1-v12.img BOOT_ITS=boot.its -j24</code>	<code>./make.sh rv1126b ../rkbin/RKBOOT/RV1126BPMINIALL.ini --spl-new</code>
RV1126B-P	Fastboot	64-bit	RO	RV1126BP EVB1	<code>./build.sh lunch:rockchip_rv1126bp_fastboot_defconfig && ./build.sh</code>	<code>./build.sh rootfs</code>	<code>make ARCH=arm64 rv1126b_defconfig rv1126b.config rv1126b-fastboot.config; make ARCH=arm64 rv1126bp-evb1-v12-fastboot-emmc.img BOOT_ITS=boot.its -j24</code>	<code>./make.sh rv1126b-emmc-fastboot rk-gpt ../rkbin/RKBOOT/RV1126BPMINIALL_FASTBOOT.ini --spl-new</code>
RV1126B-P	IPC	32-bit	RW	RV1126BP EVB1	<code>./build.sh lunch:rockchip_rv1126bp_ipc_32_evb1_v10_defconfig && ./build.sh</code>	<code>./build.sh rootfs</code>	<code>ARCH=arm rv1126b_linux_defconfig; make ARCH=arm rv1126bp-evb1-v12.img BOOT_ITS=boot.its -j24</code>	<code>./make.sh rv1126b ../rkbin/RKBOOT/RV1126BPMINIALL.ini --spl-new</code>
RV1126B-P	IPC	64-bit	RW	RV1126BP EVB1	<code>./build.sh lunch:rockchip_rv1126bp_ipc_64_evb1_v10_defconfig && ./build.sh</code>	<code>./build.sh rootfs</code>	<code>make ARCH=arm64 rv1126b_defconfig rv1126b.config; make ARCH=arm64 rv1126bp-evb1-v12.img BOOT_ITS=boot.its -j24</code>	<code>./make.sh rv1126b ../rkbin/RKBOOT/RV1126BPMINIALL.ini --spl-new</code>
RV1126B-P	Robot	64-bit	RW	RV1126BP EVB1	<code>./build.sh lunch:rockchip_rv1126bp_robot_defconfig && ./build.sh</code>	<code>./build.sh rootfs</code>	<code>make ARCH=arm64 rv1126b_defconfig rv1126b.config rv1126b_robot.config; make ARCH=arm64 rv1126bp-evb1-v12.img BOOT_ITS=boot.its -j24</code>	<code>./make.sh rv1126b ../rkbin/RKBOOT/RV1126BPMINIALL.ini --spl-new</code>

Note:

- **Rootfs Compilation:**
The default is to compile the Buildroot system. If other systems are required, set the corresponding environment variables. For example,
To compile the Buildroot system: `RK_ROOTFS_SYSTEM=buildroot ./build.sh`

To compile the Debian system: `RK_ROOTFS_SYSTEM=debian ./build.sh`

To compile the Yocto system: `RK_ROOTFS_SYSTEM=yocto ./build.sh`

- **Kernel and U-Boot Compilation:** It is necessary to specify the toolchain.
- For example, the SDK's built-in 32-bit toolchain:
`export CROSS_COMPILE=../prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi/bin/arm-none-linux-gnueabi-`
- For example, the SDK's built-in 64-bit toolchain:
`export CROSS_COMPILE=../prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu-`

8.1 Viewing SDK Compilation Commands

`make help`, for example:

```
$ make help
menuconfig          - interactive curses-based configurator
oldconfig           - resolve any unresolved symbols in .config
synconfig           - Same as oldconfig, but quietly, additionally update deps
olddefconfig        - Same as synconfig but sets new symbols to their default value
savedefconfig       - Save current config to RK_DEFCONFIG (minimal config)
...
```

The actual execution of make is `./build.sh`

You can also compile related features by running `./build.sh <target>`, and you can view the specific compilation commands through `./build.sh help`.

```
./build.sh -h
```

```
##### Rockchip Linux SDK #####
```

```
Manifest: rv1126b_linux6.1_release_v1.0.0_20250620.xml
```

```
Log colors: message notice warning error fatal
```

```
sage: build.sh [OPTIONS]
```

```
Available options:
```

```
chip[:<chip>[:<config>]]          choose chip
```

```
defconfig[:<config>]              choose defconfig
```

```
*_defconfig                       switch to specified defconfig
```

```
available defconfigs:
```

```
rockchip_defconfig
```

```
rockchip_rv1126b_evb1_v10_defconfig
```

```
rockchip_rv1126b_evb4_v10_defconfig
```

```
rockchip_rv1126b_ipc_32_evb1_v10_defconfig
```

```
rockchip_rv1126b_ipc_64_evb1_v10_defconfig
```

```
rockchip_rv1126b_robot_defconfig
```

```
rockchip_rv1126bp_evb1_v10_defconfig
```

```
rockchip_rv1126bp_ipc_32_evb1_v10_defconfig
```

rockchip_rv1126bp_ipc_64_evb1_v10_defconfig	
rockchip_rv1126bp_robot_defconfig	
olddefconfig	resolve any unresolved symbols in .config
savedefconfig	save current config to defconfig
menuconfig	interactive curses-based configurator
config	modify SDK defconfig
print-parts	print partitions
list-parts	alias of print-parts
edit-parts	edit raw partitions
misc	pack misc image
kernel-6.1[:dry-run]	build kernel 6.1
kernel[:dry-run]	build kernel
recovery-kernel[:dry-run]	build kernel for recovery
kernel-modules[:<dst dir>:dry-run]	build kernel modules
modules[:<dst dir>:dry-run]	alias of kernel-modules
linux-headers[:<arch>:dry-run]	build linux-headers
kernel-config[:dry-run]	modify kernel defconfig
kconfig[:dry-run]	alias of kernel-config
kernel-make[:<arg1>:<arg2>]	run kernel make
kmake[:<arg1>:<arg2>]	alias of kernel-make
wifibt[:<dst dir>[:<chip>]]	build Wifi/BT
amp	build and pack amp system
buildroot-config[:<config>]	modify buildroot defconfig
bconfig[:<config>]	alias of buildroot-config
buildroot-make[:<arg1>:<arg2>]	run buildroot make
bmake[:<arg1>:<arg2>]	alias of buildroot-make
buildroot-sdk	build the buildroot SDK tarball
bsdk	alias of buildroot-sdk
rootfs[:<rootfs type>]	build default rootfs
buildroot	build buildroot rootfs
yocto	build yocto rootfs
debian	build debian rootfs
recovery	build recovery
security-createkeys	create keys for security
security-misc	build misc with system encryption key
security-ramboot[:system_image]	build security ramboot
security-system[:system_image]	build security system
security-remote-sign	build remote signed image
loader[:dry-run]	build loader (u-boot)
uboot[:dry-run]	build u-boot
u-boot[:dry-run]	alias of uboot
extra-parts	pack extra partition images
firmware	pack and check firmwares
edit-package-file	edit package-file
edit-ota-package-file	edit package-file for OTA
updateimg	build update image
ota-updateimg	build update image for OTA
all	build images
release[:<subdir>[:<name>]]	release images and build info
all-release[:<subdir>[:<name>]]	build and release images
shell	setup a shell for developing
buildroot-shell	setup a shell for buildroot developing
bshell	alias of buildroot-shell

yocto-shell	setup a shell for yocto developing
yshell	alias of yocto-shell
cleanall	cleanup all
clean-misc	cleanup misc
clean-config	cleanup config
clean-recovery	cleanup recovery
clean-amp	cleanup amp
clean-kernel	cleanup kernel
clean-rootfs	cleanup rootfs
clean-firmware	cleanup firmware
clean-security	cleanup security
clean-extra-parts	cleanup extra-parts
clean-updateimg	cleanup updateimg
clean-loader	cleanup loader
post-rootfs <rootfs dir>	trigger post-rootfs hook scripts
help	display this information

Default option is '**all**'.

8.2 SDK Board-Level Configuration

Navigate to the `<SDK>/device/rockchip/rv1126b` directory:

Board-Level Configuration	Description
<code>rockchip_defconfig</code>	Default configuration (symbolic link to the generic configuration for the RV1126B EVB1 development board).
<code>rockchip_rv1126b_evb1_v10_defconfig</code>	Generic configuration for the RV1126B EVB1 board.
<code>rockchip_rv1126b_evb4_v10_defconfig</code>	Generic configuration for the RV1126B EVB4 board.
<code>rockchip_rv1126b_fastboot_defconfig</code>	Fast Boot Configuration for RV1126B-P EVB1 Board
<code>rockchip_rv1126b_ipc_32_evb1_v10_defconfig</code>	Configuration for the RV1126B EVB1 IPC (32-bit) board.
<code>rockchip_rv1126b_ipc_64_evb1_v10_defconfig</code>	Configuration for the RV1126B EVB1 IPC (64-bit) board.
<code>rockchip_rv1126b_robot_defconfig</code>	Configuration for the RV1126B EVB1 sweeping robot board.
<code>rockchip_rv1126bp_evb1_v10_defconfig</code>	Generic configuration for the RV1126B-P EVB1 board.
<code>rockchip_rv1126bp_fastboot_defconfig</code>	Fast Boot Configuration for RV1126B-P EVB1 Board.
<code>rockchip_rv1126bp_ipc_32_evb1_v10_defconfig</code>	Configuration for the RV1126B-P EVB1 IPC (32-bit) board.
<code>rockchip_rv1126bp_ipc_64_evb1_v10_defconfig</code>	Configuration for the RV1126B-P EVB1 IPC (64-bit) board.
<code>rockchip_rv1126bp_robot_defconfig</code>	Configuration for the RV1126B-P EVB1 sweeping robot board.

Configure using one of the following commands:

```
$ ./build.sh lunch
```

You're building on Linux
Lunch menu...pick a combo:

1. `rockchip_defconfig`
2. `rockchip_rv1126b_evb1_v10_defconfig`
3. `rockchip_rv1126b_evb4_v10_defconfig`
4. `rockchip_rv1126b_fastboot_defconfig`
5. `rockchip_rv1126b_ipc_32_evb1_v10_defconfig`
6. `rockchip_rv1126b_ipc_64_evb1_v10_defconfig`
7. `rockchip_rv1126b_robot_defconfig`


```
8. rockchip_rv1126bp_evb1_v10_defconfig
9. rockchip_rv1126bp_fastboot_defconfig
10. rockchip_rv1126bp_ipc_32_evb1_v10_defconfig
11. rockchip_rv1126bp_ipc_64_evb1_v10_defconfig
12. rockchip_rv1126bp_robot_defconfig
Which would you like? [1]:
```

Most customers receive the EVB1 board, which is suitable for general AIOT application scenarios.

Other feature configurations can be set through `make menuconfig` to configure related properties.

8.3 SDK Custom Configuration

The SDK can be configured through `make menuconfig`, and the main configurable components are as follows:

```
(rockchip_rv1126b_evb1_v10_defconfig) Name of defconfig to save
[*] Rootfs (Buildroot|Yocto|Debian) --->
[*] Loader (U-Boot) --->
[ ] AMP (Asymmetric Multi-Processing System)
[*] Kernel (Embedded in an Android-style boot image) --->
    Boot (Android-style boot image) --->
[*] Recovery (based on Buildroot) --->
[ ] Security feature (secureboot, encryption, verity, etc.)
    Extra partitions (oem, userdata, etc.) --->
    Firmware (partition table, misc image, etc.) --->
[*] Update (Rockchip update image) --->
[ ] Fastboot(To optimize device boot speed) --->
    Others configurations --->
```

- Rootfs: The Rootfs here represents the "root file system," where you can choose different root file system configurations such as Buildroot, Yocto, Debian, etc.
- Loader (U-Boot): This is the configuration for the bootloader, typically U-Boot, which is used to initialize the hardware and load the main operating system.
- AMPAK: A multi-core heterogeneous boot solution suitable for application scenarios requiring real-time performance.
- Kernel: Here you configure the kernel options to customize the Linux kernel according to your hardware and application needs.
- Boot: Here you configure the support format for the Boot partition.
- Recovery (based on Buildroot): This is the configuration for the recovery environment based on Buildroot, used for system recovery and upgrades.
- PCBA test (based on Buildroot): This is the configuration for a PCBA (Printed Circuit Board Assembly) test environment based on Buildroot.
- Security: Security features are enabled, including Secureboot methods, Optee storage methods, and burn-in keys.
- Extra partitions: Used to configure additional partitions.
- Firmware: Here you configure firmware-related options.

- Update (Rockchip update image): Used to configure options for the Rockchip complete firmware.
- Fastboot: Reduce system boot time in SPL, Kernel stage, and skip U-Boot stage.
- Others configurations: Other additional configuration options.

The `make menuconfig` configuration interface provides a text-based user interface for selecting and configuring various options.

After the configuration is complete, use the `make savedefconfig` command to save these configurations, so that the customized compilation will proceed according to these settings.

Through the above config, you can choose different Rootfs/Loader/Kernel configurations for various customized compilations, allowing you to flexibly select and configure system components to meet specific needs.

8.4 Fully Automatic Compilation

To ensure that each update of the Software Development Kit (SDK) proceeds smoothly, it is recommended to clean up the previous compilation artifacts before updating. This practice can prevent potential compatibility issues or compilation errors, as old compilation artifacts may not be suitable for the new version of the SDK. To clean these artifacts, you can simply run the command `./build.sh cleanall`.

Navigate to the project's root directory and execute the following command to automatically complete all compilations:

```
./build.sh all # Compiles only module code (u-Boot, kernel, Rootfs, Recovery)
               # Further execute `./build.sh ./mkfirmware.sh` for firmware packaging

./build.sh     # Compiles module code (u-Boot, kernel, Rootfs, Recovery)
               # Packages into a complete update.img upgrade package
               # All compilation information is copied and generated in the out directory
```

The default is Buildroot, but you can specify a different rootfs by setting the environment variable `RK_ROOTFS_SYSTEM`. `RK_ROOTFS_SYSTEM` currently supports three systems: buildroot, debian, and yocto.

For example, to generate a Debian system, you can use the following commands:

```
export RK_ROOTFS_SYSTEM=debian
./build.sh
or
RK_ROOTFS_SYSTEM=debian ./build.sh
```

8.5 Module Compilation

8.5.1 U-Boot Compilation

```
./build.sh uboot
```

8.5.2 Kernel Compilation

- Method One

```
./build.sh kernel
```

- Method Two

```
cd kernel
export CROSS_COMPILE=../prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-
none-linux-gnu/bin/aarch64-none-linux-gnu-
make ARCH=arm64 rockchip_linux_defconfig rv1126b.config
make ARCH=arm64 rv1126b-evb1-v10.img BOOT_ITS=boot.its -j4
```

Note: `rv1126b-evb1-v10` can be replaced with the specific hardware DTS

- Method Three

```
cd kernel
export CROSS_COMPILE=aarch64-linux-gnu-
make ARCH=arm64 rockchip_linux_defconfig rv1126b.config
make ARCH=arm64 rv1126b-evb1-v10.img BOOT_ITS=boot.its -j4
```

Note: `rv1126b-evb1-v10` can be replaced with the specific hardware DTS

8.5.3 Recovery Compilation

Navigate to the root directory of the project and execute the following command to automatically complete the compilation and packaging of Recovery.

```
<SDK># ./build.sh recovery
```

After compilation, the recovery.img is generated in the Buildroot directory
`output/rockchip_rv1126b_recovery/images`.

Note: The recovery.img includes the kernel, so every time the Kernel is modified, Recovery needs to be repackaged and generated. The method to repackage Recovery is as follows:

```
<SDK># source buildroot/envsetup.sh
<SDK># cd buildroot
<SDK># make recovery-reconfigure
<SDK># cd -
<SDK># ./build.sh recovery
```

Note: Recovery is a non-essential feature, and some board-level configurations may not set it up.

8.5.4 Buildroot Compilation

Navigate to the root directory of the project and execute the following command to automatically complete the compilation and packaging of the Rootfs:

```
./build.sh rootfs
```

After the compilation, different formats of the image are generated in the `output/rockchip_rv1126b/images` directory under the Buildroot directory, with the default format being `rootfs.ext4`.

For specific details, refer to the Buildroot development documentation:

```
<SDK>/docs/en/Linux/System/Rockchip_Developer_Guide_Buildroot_EN.pdf
```

8.5.4.1 Buildroot Module Compilation

Set up configurations for different chips and target features via `source buildroot/envsetup.sh`

```
$ source buildroot/envsetup.sh
Top of tree: rv1126b

You're building on Linux
Lunch menu...pick a combo:

65. rockchip_rv1126b
66. rockchip_rv1126b_recovery
...

Which would you like? [1]:
```

The default selection is 66, `rockchip_rv1126b`. Then proceed to the RV1126B's Buildroot directory to start compiling the relevant modules.

The `rockchip_rv1126b_recovery` is used for compiling the Recovery module.

For example, to compile the rockchip-test module, the common compilation commands are as follows:

Navigate to the buildroot directory

```
<SDK># cd buildroot
```

- Delete and recompile rockchip-test

```
buildroot# make rockchip-test-recreate
```

- Rebuild rockchip-test

```
buildroot# make rockchip-test-rebuild
```

- Delete rockchip-test

```
buildroot# make rockchip-test-dirclean  
or  
buildroot# rm -rf output/rockchip_rv1126b/build/rockchip-test-master/
```

8.5.5 Debian Compilation

```
./build.sh debian
```

After compilation, the `linaro-rootfs.img` is generated in the `debian` directory.

Note: It is necessary to install the relevant dependency packages in advance.

```
sudo apt-get install binfmt-support qemu-user-static live-build  
sudo dpkg -i ubuntu-build-service/packages/*  
sudo apt-get install -f
```

For specific details, please refer to the Debian development documentation:

```
<SDK>/docs/en/Linux/System/Rockchip_Developer_Guide_Debian_EN.pdf
```

8.5.6 Yocto Compilation

Navigate to the root directory of the project and execute the following command to automatically complete the compilation and packaging of the Rootfs:

```
./build.sh yocto
```

After compilation, the `rootfs.img` is generated in the directory `yocto/build/latest`.

The default user login is root. For more information on Yocto, please refer to the [Rockchip Wiki](#).

FAQ:

- If you encounter the following issue during the compilation:

```
Please use a locale setting which supports UTF-8 (such as LANG=en_US.UTF-8).  
Python can't change the filesystem locale after loading so we need a UTF-8  
when Python starts or things won't work.
```

Solution:

```
locale-gen en_US.UTF-8
export LANG=en_US.UTF-8 LANGUAGE=en_US.en LC_ALL=en_US.UTF-8
```

Or refer to [setup-locale-python3](#)

- If you encounter git permission issues that cause compilation errors.

Due to the addition of the CVE-2022-39253 security detection patch in the newer version of git, if an older version of Yocto is required, poky must include the following to be fixed:

```
commit ac3eb2418aa91e85c39560913c1ddfd2134555ba
Author: Robert Yang <liezhi.yang@windriver.com>
Date:   Fri Mar 24 00:09:02 2023 -0700

    bitbake: fetch/git: Fix local clone url to make it work with repo

    The "git clone /path/to/git/objects_symlink" couldn't work after the following
    change:

    https://github.com/git/git/commit/6f054f9fb3a501c35b55c65e547a244f14c38d56
```

Alternatively, you can also downgrade the PC's git version to V2.38 or an earlier version, for example:

```
$ sudo apt update && sudo apt install -y libcurl4-gnutls-dev

git clone https://gitee.com/mirrors/git.git --depth 1 -b v2.38.0
cd git
make git -j8
make install
sudo install -m 0755 git /usr/bin/git
```

8.6 Firmware Packaging

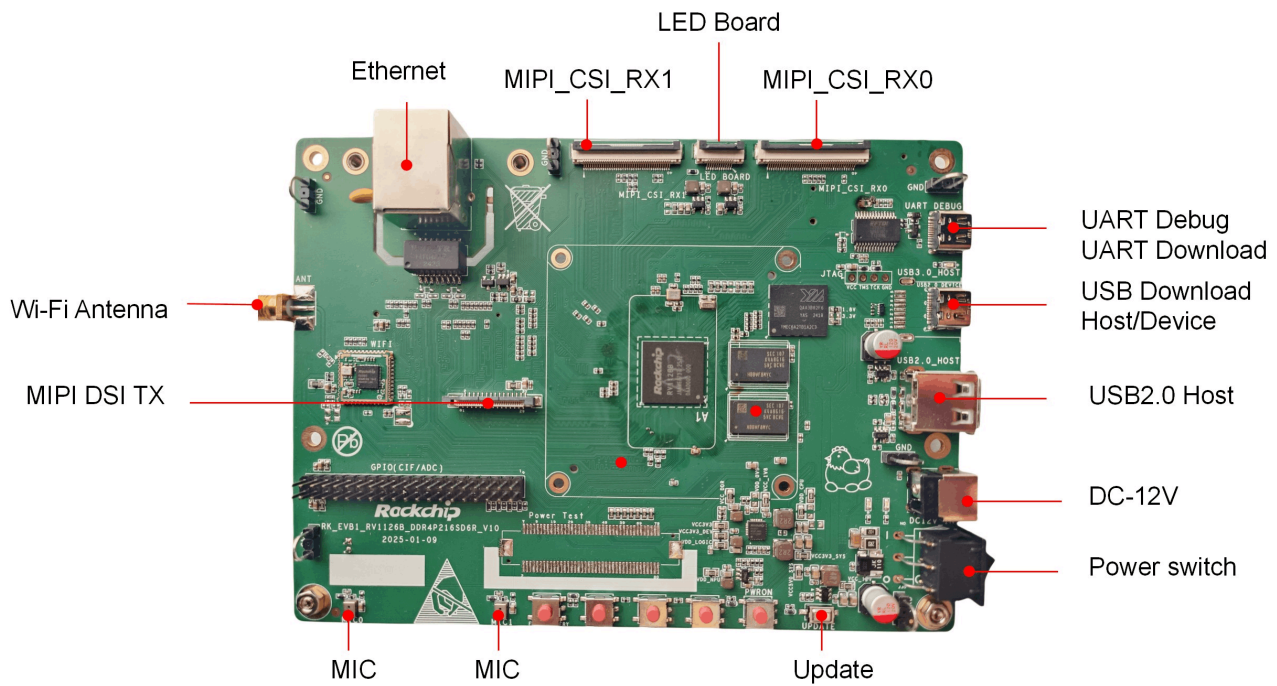
After compiling the various components such as Kernel, U-Boot, Recovery, and Rootfs, navigate to the root directory of the project and execute the following command to automatically complete the packaging of all firmware into the `output/firmware` directory:

Firmware Generation:

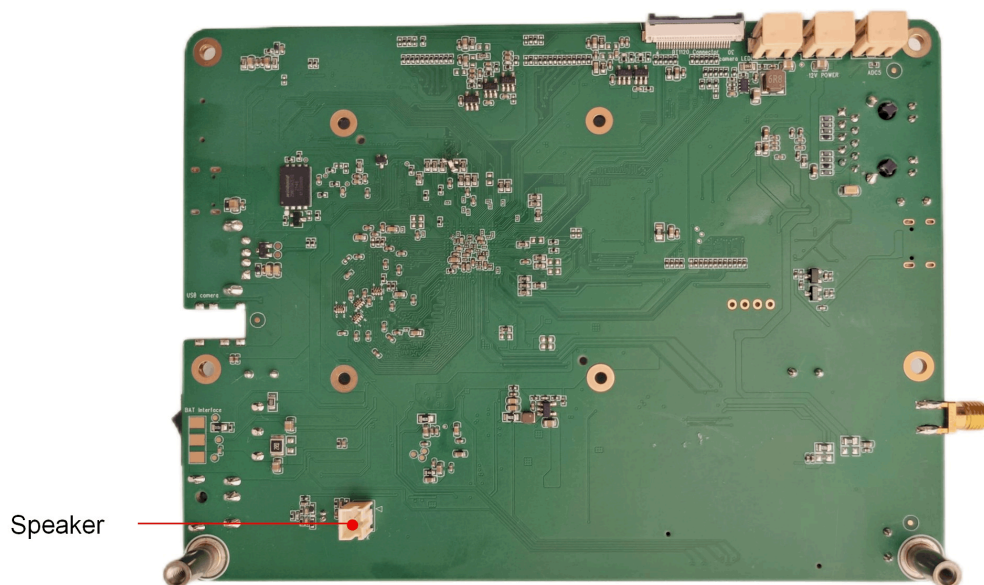
```
./build.sh firmware
```

9. Flashing Instructions

The front interface layout of the RV1126B EVB1 V10 Development Board is shown below:



The bottom interface layout of the RV1126BP EVB1 V10 Development Board is shown below:



9.1 Windows Flashing Instructions

The SDK provides a Windows flashing tool (the tool version must be V3.37 or above), located in the root directory of the project:

```
tools/
├── windows/RKDevTool
```

As shown in the figure below, after compiling the corresponding firmware, the device needs to enter the MASKROM or BootROM flashing mode. Connect the USB download cable, hold the "MASKROM" button and press the reset button "RST" without releasing, then release to enter the MASKROM mode. Load the corresponding path of the compiled firmware and click "Execute" to flash. Alternatively, hold the "recovery" button without releasing and press the reset button "RST", then release to enter the loader mode for flashing. Below are the partition offsets for MASKROM mode and the flashing files.

(Note: The Windows PC needs to run the tool with administrator privileges to execute)

Download Image

Upgrade Firmware

Advanced Function

#	☐	Storage	Address	Name	Path	...
1	☒		0x00000000	loader	..\rockdev\MiniLoaderAll.bin	
2	☒		0x00000000	parameter	..\rockdev\parameter.txt	
3	☒		0x00004000	uboot	..\rockdev\uboot.img	
4	☒		0x00006000	misc	..\rockdev\misc.img	
5	☒		0x00008000	boot	..\rockdev\boot.img	
6	☒		0x00028000	recovery	..\rockdev\recovery.img	
7	☐		0x00068000	backup		
8	☒		0x01C78000	oem	..\rockdev\oem.img	
9	☒		0x00078000	rootfs	..\rockdev\rootfs.img	
10	☒		0x01D18000	userdata	..\rockdev\userdata.img	

Loader:

Run

Switch

Dev Partition

Clear

☐ Write by Address

Found One MASKROM Device

1-1 :MASKROM

Note: Before flashing, the latest USB driver must be installed. For more details on the driver, see:

```
<SDK>/tools/windows/DriverAssitant_v5.13.zip
```

9.2 Linux Flashing Instructions

The flashing tool for Linux is located in the `tools/linux` directory (the `Linux_Upgrade_Tool` version must be V2.36 or above). Please ensure that your board is connected to MASKROM/loader rockusb. For example, if the firmware compiled is in the `rockdev` directory, the upgrade commands are as follows:


```

sudo ./upgrade_tool ul rockdev/MiniLoaderAll.bin -noreset
sudo ./upgrade_tool di -p rockdev/parameter.txt
sudo ./upgrade_tool di -u rockdev/uboot.img
sudo ./upgrade_tool di -misc rockdev/misc.img
sudo ./upgrade_tool di -b rockdev/boot.img
sudo ./upgrade_tool di -recovery rockdev/recovery.img
sudo ./upgrade_tool di -oem rockdev/oem.img
sudo ./upgrade_tool di -rootfs rockdev/rootfs.img
sudo ./upgrade_tool di -userdata rockdev/userdata.img
sudo ./upgrade_tool rd

```

Or upgrade the complete firmware after packaging:

```

sudo ./upgrade_tool uf rockdev/update.img

```

Or upgrade in the root directory when the machine is running in MASKROM state:

```

./rkflash.sh

```

9.3 System Partition Description

Default Partition Description (Below is the RV1126B EVB Partition Reference)

Number	Start	End	Size	Name
1	8389kB	12.6MB	4194kB	uboot
2	12.6MB	16.8MB	4194kB	misc
3	16.8MB	83.9MB	67.1MB	boot
4	83.9MB	218MB	134MB	recovery
5	218MB	252MB	33.6MB	bakcup
6	252MB	6694MB	6442MB	rootfs
7	6694MB	6828MB	134MB	oem
8	6828MB	31.3GB	24.4GB	userdata

- uboot Partition: For the uboot.img compiled from uboot.
- misc Partition: For misc.img, used by recovery.
- boot Partition: For boot.img compiled from the kernel.
- recovery Partition: For recovery.img compiled by recovery.
- backup Partition: Reserved, not in use at the moment.
- rootfs Partition: For rootfs.img compiled by buildroot, debian, or yocto.
- oem Partition: For manufacturers to use, storing manufacturer's apps or data. Mounted at the /oem

directory.

- userdata Partition: For apps to temporarily generate files or for end users, mounted under the /userdata directory.

10. Frequently Asked Questions (FAQ)

10.1 How to Enable DDR Frequency Scaling

The DDR frequency scaling feature is not enabled by default on the EVB. If you need to enable it, take EVB1 as an example, you can enable it through the following configuration:

```
--- a/arch/arm64/boot/dts/rockchip/rv1126b-evb.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rv1126b-evb.dtsi
@@ -120,6 +120,15 @@ vcc12v_dcin: vcc12v-dcin {
    };
};

+&dfi {
+    status = "okay";
+};
+
+&dmc {
+    center-supply = <&vdd_logic>;
+    status = "okay";
+};
```

configure defconfig:

```
+ CONFIG_ARM_ROCKCHIP_DMC_DEVFREQ=y
+ CONFIG_PM_DEVFREQ_EVENT=y
```

```
# Check the configurable range of DDR frequencies
cat /sys/class/devfreq/dmc/available_frequencies
# Set DDR governors to userspace mode
echo userspace > /sys/class/devfreq/dmc/governor
# Set DDR frequency
echo 780000000 > /sys/class/devfreq/dmc/userspace/set_freq
# Check current DDR frequency
cat /sys/class/devfreq/dmc/cur_freq
```

10.2 Issue with RV1126BP Read/Write

In the Linux system, to determine the file system type and whether it is readable/writable, you can specify relevant parameters in the Kernel cmdline. For example, if the system is in ext4 format and is readable/writable, you can make the following settings in the following location:

```
--- a/arch/arm64/boot/dts/rockchip/rv1126bp-evb.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rv1126bp-evb.dtsi
@@ -8,7 +8,7 @@

 / {
     chosen {
-        bootargs = "earlycon=uart8250,mmio32,0x20810000 console=ttyFIQ0
root=PARTUUID=614e0000-0000 rootwait snd_soc_core.prealloc_buffer_size_kbytes=16
coherent_pool=32K";
+        bootargs = "earlycon=uart8250,mmio32,0x20810000 console=ttyFIQ0 rw
root=PARTUUID=614e0000-0000 rootfstype=ext4 rootwait
snd_soc_core.prealloc_buffer_size_kbytes=16 coherent_pool=32K";
```

10.3 How to Read CPU Serial Number

CPU Serial Number (i.e., CPU ID).

```
cat /sys/module/rockchip_cpufreq/parameters/id
```